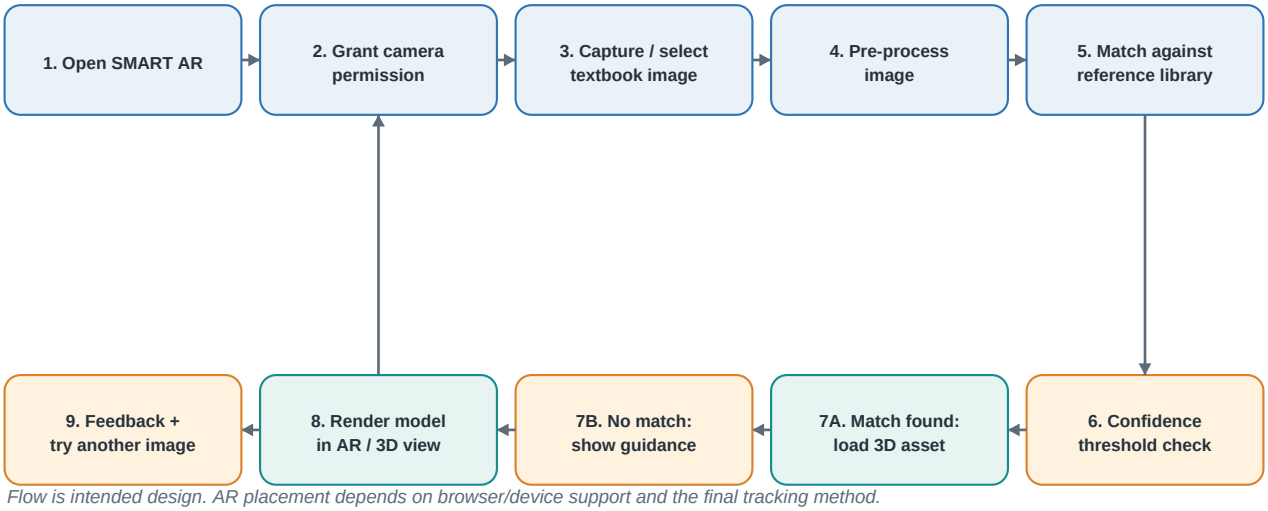
SMART AR

AI-Assisted Image Recognition & Markerless Augmented Reality for Interactive Learning



Flow is intended design. AR placement depends on browser/device support and the final tracking method.

Technical Project Documentation

Team: Team Spartan

Team ID: HSC|TN|02851

Context: Educational technology project for students in Tamil Nadu, India

Purpose: document the proposed system, end-to-end workflow, architecture, recognition approach, asset mapping, testing plan, risks, and implementation roadmap.

Status note: This document describes the intended design. Confirmed implementation details should be updated as the prototype is built and tested.

1. Project Overview

SMART AR is a web-based learning concept that aims to connect textbook images with interactive 3D visualizations. A learner scans or selects an image, the system compares it with a prepared reference-image library, and a successful match retrieves the linked 3D asset. The model is then displayed in a 3D/AR view where supported by the device and browser.

Problem addressed

- Abstract concepts can be difficult to understand through text and flat diagrams alone.
- Some AR learning tools require printed markers, dedicated apps, or costly content creation.
- Teachers and students need a lightweight way to explore reusable 3D learning assets.

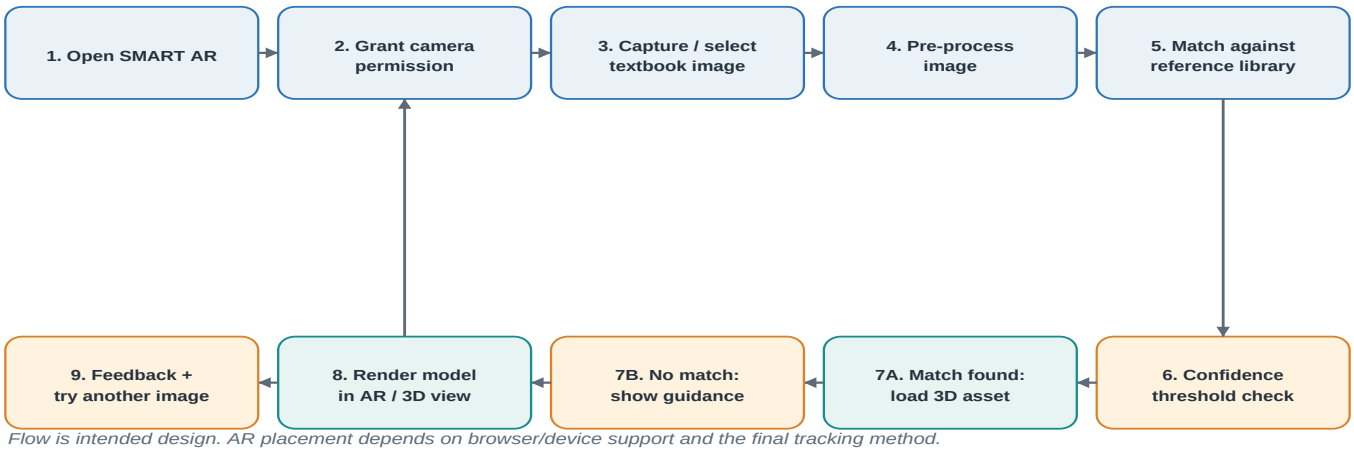
Goals

- Make the learning flow simple: scan → recognize → load model → explore.
- Reuse a curated library of existing 3D assets rather than creating every model from scratch.
- Use image matching to select a model and provide a clear fallback when no suitable match is found.
- Keep the design modular so more subjects, images, and 3D models can be added later.

Target users: school and college students, educators, and institutions looking for accessible visual-learning resources.

Alignment: UN SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure. These are intended impact alignments, not a claim of measured outcomes.

2. End-to-End User Workflow



Flow is intended design. AR placement depends on browser/device support and the final tracking method.

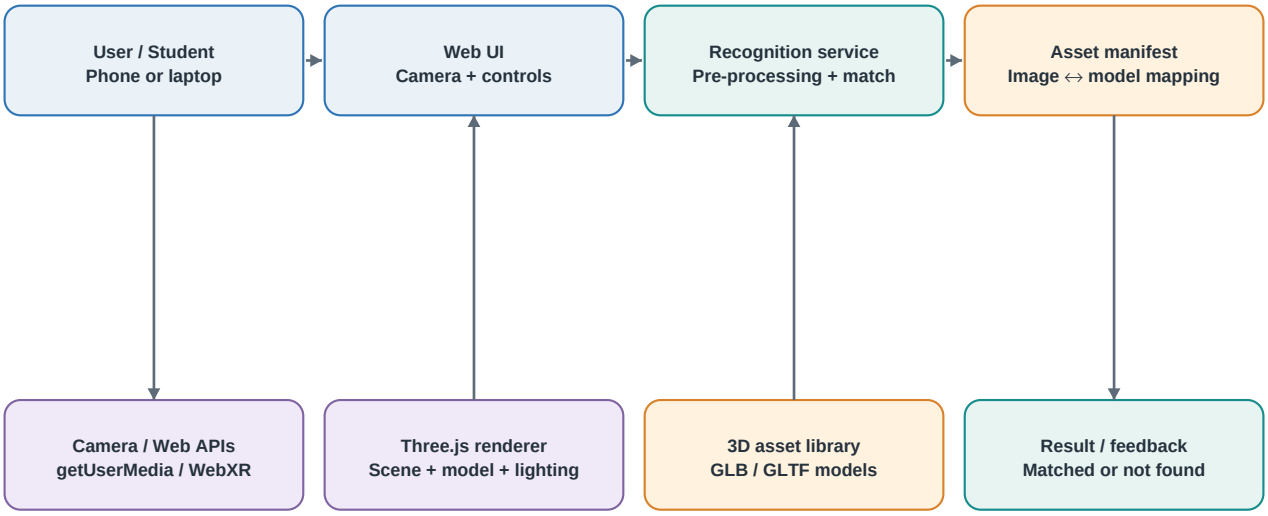
Workflow explanation

Step	System action	Expected result
1–2. Start	Open the web app and request camera permission, or offer image upload where available.	User can provide an image input.
3–4. Capture & prepare	Capture/select an image; resize and normalize it for comparison.	A consistent image is ready for matching.
5–6. Recognize	Compare image features against reference images and evaluate a confidence/quality score.	Candidate match accepted or rejected.

Step	System action	Expected result
7. Resolve asset	Use the asset manifest to map the accepted image ID to a model file and metadata.	Correct model is selected, if a match exists.
8–9. Visualize	Load the model, display it in a 3D viewer or AR mode, and offer reset/retry/help controls.	Student explores the model or receives guidance to try again.

3. System Architecture

The architecture separates the user interface, image-recognition logic, asset mapping, 3D rendering, and feedback. This separation makes it easier to replace or improve the recognition algorithm without rebuilding the entire learning interface.



Conceptual architecture; recognition may run in-browser or behind an API depending on implementation.

Main components

Component	Responsibilities	Example technologies / artifacts
Web interface	Start scan, capture/upload image, show progress, display match/no-match messages, controls.	HTML, CSS, JavaScript
Camera & device access	Request camera access; handle permission denial; check device/browser capability.	getUserMedia; WebXR where supported
Recognition module	Prepare input, extract features, compare with reference images, score and return best candidate.	OpenCV.js or Python/OpenCV service; ORB as a possible baseline
Asset manifest	Map stable image IDs to model path, title, subject, scale and metadata.	JSON manifest or database
3D/AR renderer	Load model, set camera/light/scale, enable rotate/zoom and AR placement where supported.	Three.js; GLB/GLTF; WebXR if available
Asset library	Store approved 3D models and reference images with license/source information.	Static files or hosted asset storage

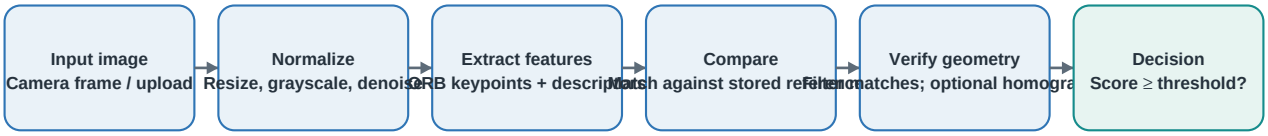
Deployment options

- **Browser-only prototype:** run image matching in the browser if performance and library size allow.
- **Client + recognition API:** send an image to a backend service for matching and return the selected asset ID. This may simplify model updates but needs network access and privacy safeguards.
- **Static hosting:** host the interface and model files on a static site; use a backend only if needed for recognition, accounts, analytics, or content management.

The diagram is a conceptual target architecture. The final deployment should reflect the code and services actually implemented.

4. Image Recognition and Asset Matching

The core matching task is to determine whether the input image corresponds to one of the known reference images. A practical first version can use feature-based matching; a later version may add learned embeddings or a trained classifier if the dataset and evaluation results justify it.



Possible ORB-based baseline

- Prepare a reference set: store one or more reference images for each supported learning concept.
- Extract ORB keypoints and descriptors from each reference image during asset-library preparation; cache them if practical.
- For a captured image, extract descriptors and compare them with reference descriptors using a suitable matcher.
- Filter weak matches and optionally verify geometric consistency with a homography when the image is planar and the conditions are appropriate.
- Calculate a match score using evidence such as good-match count, inlier ratio, and image quality; accept only if the score passes a validated threshold.
- Return a stable concept/image ID rather than directly hard-coding a model path in the recognition code.

Important limitation

ORB is a candidate approach, not a guarantee of robust recognition. Lighting, blur, page curvature, perspective, occlusion, similar diagrams, and low-texture images can reduce accuracy. Thresholds must be tuned using representative test images. If the source image is not planar or matching becomes unreliable, evaluate alternative feature or embedding-based methods.

Asset manifest example

Field	Example value	Purpose
image_id	bio_heart_001	Unique key returned by matcher.
display_name	Human heart	Label shown to the learner.
reference_images	references/heart_front.jpg	Images used to recognize the concept.
model_path	models/heart.glb	3D model to load after a match.
subject	Biology	Filter and organize learning content.
license_source	Asset source and license details	Track permitted use and attribution.

No-match behaviour

If confidence is too low, do not show a potentially incorrect model. Explain that the image was not recognized and suggest: improve lighting, hold the page steady, frame one diagram, try another angle, or choose an image from the supported set. A future version can let an administrator add a new reference image and link it to an approved model.

5. 3D Visualization and AR Experience

After the recognizer returns an accepted image ID, the asset resolver finds the corresponding model metadata. The renderer loads the model and provides a usable viewing experience. AR placement is an additional device-dependent capability, not the same as recognizing the textbook image.

Model loading sequence

- Read the asset manifest entry for the matched image ID.
- Check that the model URL exists and the file type is supported.
- Load the GLB/GLTF model with Three.js and show a loading indicator.
- Apply a default scale, orientation, camera position, lighting, and material settings.
- Offer rotate, zoom, reset, and return-to-scan controls.
- If supported, offer an AR mode using a compatible WebXR flow; otherwise keep the interactive 3D viewer available.

Terminology to keep clear

Capability	Meaning	Design note
Image recognition	Identifies which known textbook image or diagram was supplied.	Uses reference images and matching logic.
Markerless tracking	Places or maintains virtual content in the real environment without a printed fiducial marker, using supported tracking capabilities.	Requires compatible platform/browser support; recognition alone does not provide world tracking.
3D viewer	Displays and lets the learner manipulate a 3D model on screen.	Useful fallback when AR APIs are unavailable.
WebXR AR	Browser interface to supported immersive AR features.	Check device/browser support and permissions; do not promise universal compatibility.

Suggested user interface states

- Idle: Scan image / Upload image
- Permission needed: explain why camera access is required
- Processing: show a short progress indicator
- Match found: show concept title and open 3D model
- No match: provide actionable recapture tips
- Model loading failed: show retry and return-to-scan controls
- AR unavailable: allow continued use in 3D viewer

6. Proposed Technology Stack and Repository Structure

Layer	Recommended option	Role
Frontend	HTML, CSS, JavaScript	Responsive interface and app states.
3D rendering	Three.js + GLB/GLTF assets	Load and render interactive models.
AR capability	WebXR where supported	Optional immersive AR mode on compatible devices.
Image matching	OpenCV.js or Python + OpenCV; ORB as baseline	Feature extraction, matching and score calculation.
Metadata	JSON manifest initially; database if needed	Map reference images to models and metadata.
Hosting	Static hosting plus optional API hosting	Serve frontend, reference data, models and optional recognition endpoint.

Illustrative repository structure

```
smart-ar/
■■■ index.html # app entry point
■■■ src/
■ ■■■ ui/ # screens and controls
■ ■■■ camera/ # camera and image input
■ ■■■ recognition/ # preprocessing and matching
■ ■■■ assets/ # manifest and asset resolver
■ ■■■ viewer/ # Three.js scene and AR mode
■■■ data/references/ # approved reference images
■■■ public/models/ # GLB/GLTF models
■■■ tests/ # matching, mapping, and UI tests
■■■ docs/architecture.md
■■■ README.md
■■■ LICENSE
```

This structure is a recommendation; adjust it to match the actual project repository and framework.

7. Data, Privacy, Accessibility and Sustainability

- **Data minimization:** process images locally where feasible. If images are sent to a server, explain this to users and avoid retaining images unless needed and consented to.
- **Permissions:** request camera access only when the user starts scanning; handle denied permissions gracefully.
- **Asset licensing:** track the source, creator, and license for each model and reference image. Use assets only under their permitted terms.
- **Accessibility:** provide text labels, clear feedback, readable contrast, and a non-AR 3D-view fallback.
- **Low-bandwidth use:** compress models, load assets on demand, cache stable files when appropriate, and provide clear loading/error states.
- **Sustainability:** reuse digital assets across topics and courses; prioritize maintainable, modular content rather than recreating assets for every lesson.

8. Testing Plan and Success Metrics

Evaluate the prototype with a defined test set before claiming recognition accuracy or educational impact. Separate technical performance from learner outcomes.

Test area	Test cases	Measure / pass evidence
Recognition	Supported images under varied angles, lighting, distance, blur and backgrounds.	Precision/recall or correct-match rate on held-out images; false-match count.
No-match handling	Unknown images, unrelated pages, similar diagrams.	Unknown inputs rejected safely; helpful recovery message displayed.
Asset mapping	Every known image ID; missing or malformed model path.	Correct model selected; broken references handled gracefully.
Rendering	Large/small models, loading failures, rotation, zoom, reset.	No app crash; usable controls; load time recorded.
Device compatibility	Supported and unsupported browsers/devices; denied camera permission.	Capability checks and fallbacks behave as expected.
Usability	Students/educators complete scan-to-model task.	Task completion rate, time-on-task, user feedback.

Suggested KPIs to report

- Recognition accuracy on a held-out test set (report sample size and conditions).
- False-match rate and no-match rejection rate.
- Median time from image submission to result.
- 3D model load success rate and load time.
- Percentage of test users who complete the scan-to-model task without assistance.
- User-reported clarity/usefulness of the 3D visualization.

Do not enter invented figures. Populate these metrics after running tests on the actual prototype.

9. Risks and Mitigation

Risk	Why it matters	Mitigation
Similar images cause false match	Wrong model can confuse learning.	Use confidence thresholds, geometric checks, multiple references, and unknown-image tests.
Lighting/perspective variation	Camera image differs from stored reference.	Collect varied examples, add capture guidance, and test robustness.
AR browser incompatibility	Some devices do not support the required APIs.	Feature detection and interactive 3D fallback.
Large model downloads	Slow load on mobile or low bandwidth.	Optimize assets and load only the selected model.
Asset copyright/license issue	Unapproved content may block distribution.	Keep source/license metadata and use permitted assets.
Camera privacy concern	Users may be uncomfortable sharing images.	Minimize collection, explain processing, secure transfers and avoid unnecessary retention.

10. Implementation Roadmap

Phase	Deliverables	Exit check
1. Scope & assets	Choose a small subject set; collect permitted reference images and 3D models; define manifest.	Every supported image maps to one valid asset entry.
2. Web viewer	Build responsive UI and a Three.js model viewer with controls.	Models load and can be explored without AR.
3. Image matching	Implement baseline matching and confidence/no-match logic.	Test set includes known and unknown images; results recorded.
4. Connect pipeline	Wire image input → matcher → asset resolver → viewer.	End-to-end scan-to-model flow works for supported examples.
5. AR capability	Add WebXR or compatible AR flow after checking target-device support.	Supported devices enter AR; others use 3D fallback.
6. Validate & document	Run tests, measure KPIs, document limitations, setup, architecture and demo.	Reproducible setup and evidence-based results are published.

11. Sustainability and Future Enhancements

- Add new learning topics by registering approved reference images and linking them to existing 3D models.
- Create a teacher/admin workflow to validate content and manage asset metadata.
- Evaluate ML-based image embeddings if feature-based matching is not robust enough.
- Add multilingual descriptions, short learning prompts, and accessibility improvements.
- Use learner and educator feedback to prioritize content; collect only necessary, consented analytics.

12. References and Useful Documentation

- Three.js documentation: https://threejs.org/docs/
- MDN WebXR Device API: https://developer.mozilla.org/en-US/docs/Web/API/WebXR_Device_API
- MDN MediaDevices.getUserMedia: https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia
- OpenCV documentation: https://docs.opencv.org/
- Khronos glTF / GLB specification: https://www.khronos.org/gltf/
- United Nations Sustainable Development Goals: https://sdgs.un.org/goals

Final note: This document is a technical design guide for SMART AR, not proof that every listed feature is already implemented. Before sharing as a final project report, update the technology stack, architecture, repository link, screenshots, test results, and implementation status to match the actual build.